MNIST Handwritten Digit Recognition

Transfer Learning Using MobileNetV2

1. Project Overview

This project uses the MNIST handwritten digit dataset and a pre-trained MobileNetV2 model to classify digits from 0 to 9. The model was trained and fine-tuned using TensorFlow and Keras.

2. Model Accuracy Results

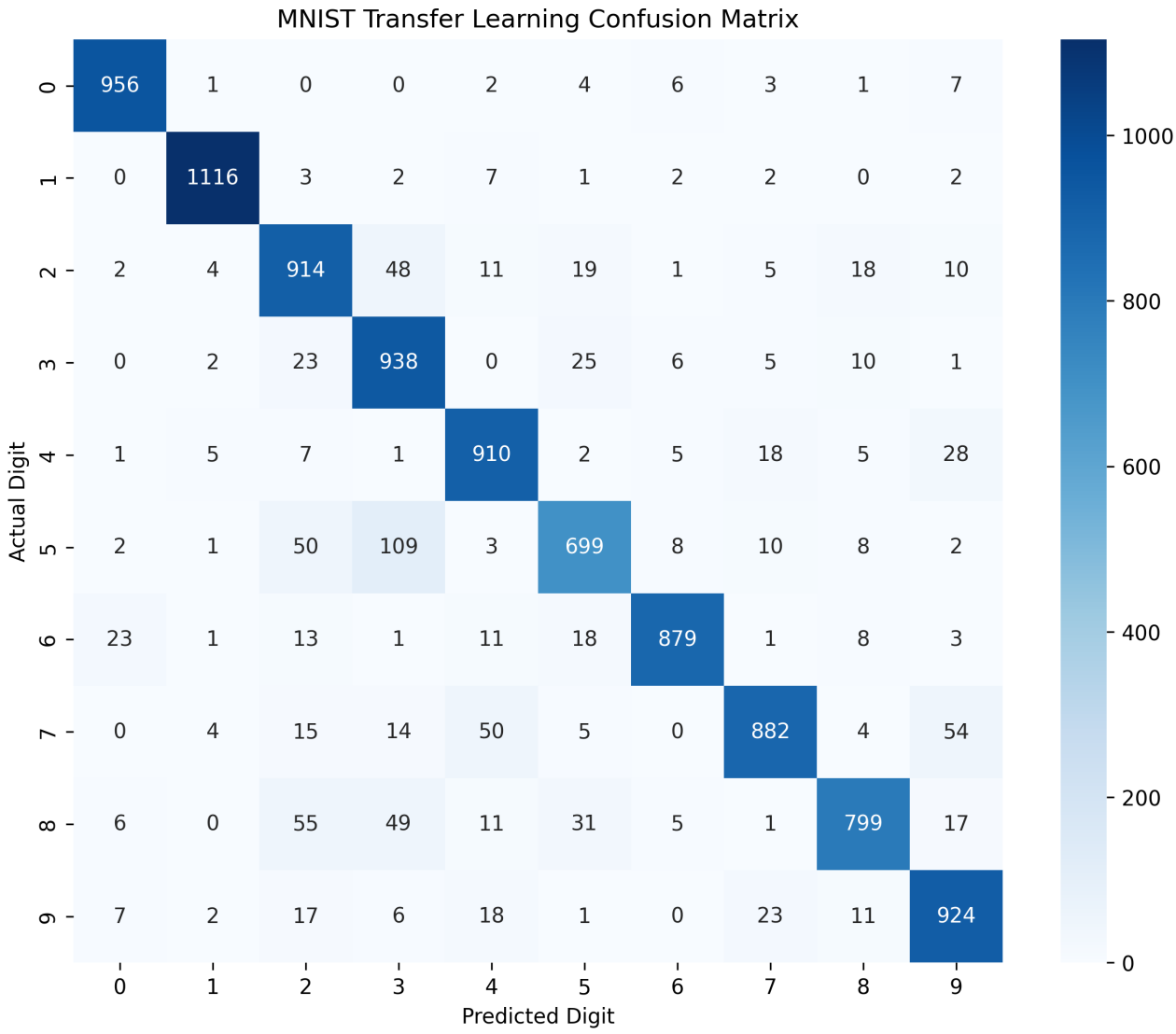
Initial model test accuracy: 69.19%

Fine-tuned model test accuracy: approximately 90%

Improvement: approximately 20.81 percentage points.

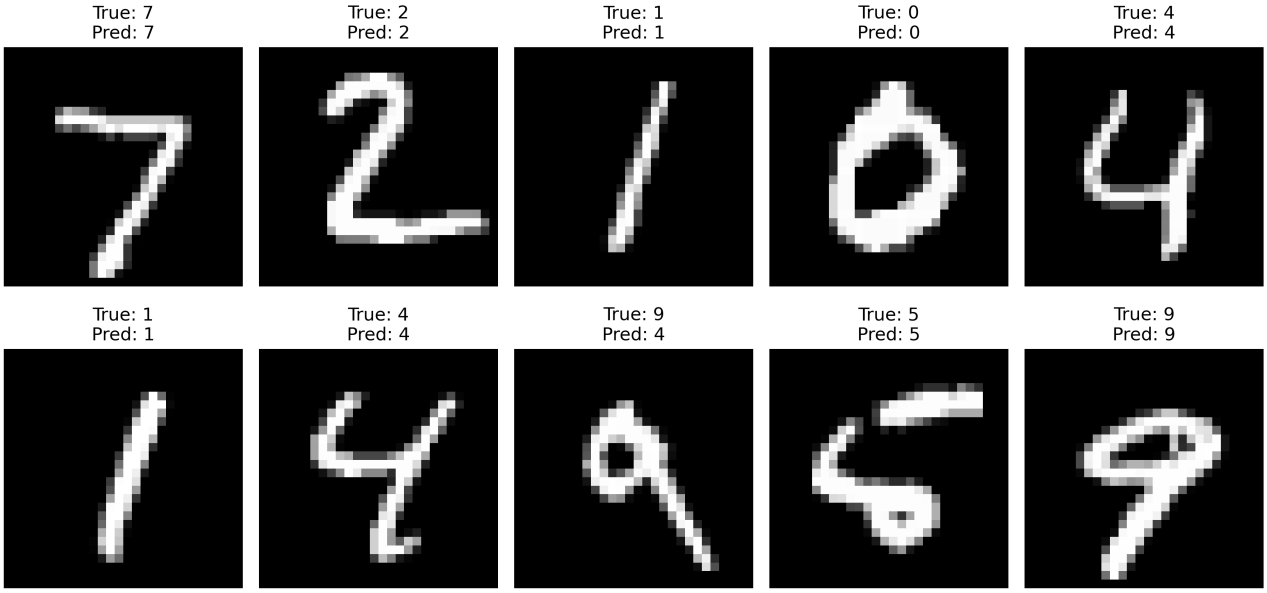
3. Confusion Matrix

The confusion matrix compares actual MNIST labels with predicted labels and reveals which digits are most frequently confused.


MNIST Transfer Learning Confusion Matrix
Actual Digit
Predicted Digit
0 1 2 3 4 5 6 7 8 9
956 1 0 0 2 4 6 3 1 7
0 1116 3 2 7 1 2 2 0 2
2 4 914 48 11 19 1 5 18 10
0 2 23 938 0 25 6 5 10 1
1 5 7 1 910 2 5 18 5 28
2 1 50 109 3 699 8 10 8 2
23 1 13 1 11 18 879 1 8 3
0 4 15 14 50 5 0 882 4 54
6 0 55 49 11 31 5 1 799 17
7 2 17 6 18 1 0 23 11 924
1000
800
600
400
200
0

4. Sample Digit Predictions

The following images show handwritten MNIST digits, their actual labels and the model's predictions.



5. Python Source Code

mnist_transfer_learning.py

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model

# Step 8: Load the MNIST dataset

(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Display the dataset shapes
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)

# Step 9: Convert grayscale images to RGB

x_train = np.stack([x_train] * 3, axis=-1)
x_test = np.stack([x_test] * 3, axis=-1)

print("RGB training images:", x_train.shape)
print("RGB testing images:", x_test.shape)

# Step 10: Resize images to 32 x 32 pixels

x_train = tf.image.resize(
    tf.convert_to_tensor(x_train), [32, 32]
)

x_test = tf.image.resize(
    tf.convert_to_tensor(x_test), [32, 32]
)

print("Resized training images:", x_train.shape)
print("Resized testing images:", x_test.shape)

# Step 11: Normalise pixel values

x_train = x_train / 255.0
x_test = x_test / 255.0

print("Minimum pixel value:", tf.reduce_min(x_train).numpy())
print("Maximum pixel value:", tf.reduce_max(x_train).numpy())

# Step 12: Convert labels to categorical format

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)

print("Example encoded label:", y_train[0])

# Step 13: Load the pre-trained MobileNetV2 model

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(32, 32, 3)
)

# Freeze the pre-trained layers
base_model.trainable = False

# Display model summary
base_model.summary()
# Step 14: Add new classification layers
```

```
x = base_model.output

x = GlobalAveragePooling2D()(x)

x = Dense(128, activation='relu')(x)

predictions = Dense(10, activation='softmax')(x)

# Create the final transfer learning model
model = Model(
    inputs=base_model.input,
    outputs=predictions
)

# Display the complete model
model.summary()

# Step 14: Add new classification layers

x = base_model.output

x = GlobalAveragePooling2D()(x)

x = Dense(128, activation='relu')(x)

predictions = Dense(10, activation='softmax')(x)

# Create the final transfer learning model
model = Model(
    inputs=base_model.input,
    outputs=predictions
)

# Display the complete model

model.summary()

# Step 14: Add new classification layers

x = base_model.output

x = GlobalAveragePooling2D()(x)

x = Dense(128, activation='relu')(x)

predictions = Dense(10, activation='softmax')(x)

# Create the final transfer learning model
model = Model(
    inputs=base_model.input,
    outputs=predictions
)

# Display the complete model
model.summary()

# Step 14: Add new classification layers

x = base_model.output

x = GlobalAveragePooling2D()(x)

x = Dense(128, activation='relu')(x)

predictions = Dense(10, activation='softmax')(x)

# Create the final transfer learning model
model = Model(
    inputs=base_model.input,
    outputs=predictions
)

# Display the complete model
model.summary()

# Step 15: Compile the transfer learning model

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
print("Model compiled successfully!")

# Step 16: Preprocess images for MobileNetV2

# Convert pixel values from [0, 1] to [-1, 1]
x_train = (x_train * 2.0) - 1.0
x_test = (x_test * 2.0) - 1.0

print("Minimum pixel value:", tf.reduce_min(x_train).numpy())
print("Maximum pixel value:", tf.reduce_max(x_train).numpy())

print("Images ready for MobileNetV2 training!")

# Step 17: Train the transfer learning model

history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_data=(x_test, y_test)
)

print("Model training completed!")

# Step 18: Evaluate the trained model

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=1
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

print(f"MNIST Test Accuracy: {test_accuracy * 100:.2f}%")

# Step 19: Save the trained MNIST model

model.save("mnist_transfer_model.keras")

print("Model saved successfully!")

# Step 20: Unfreeze some MobileNetV2 layers

# Allow the base model to be fine-tuned
base_model.trainable = True

# Keep the earlier layers frozen
for layer in base_model.layers[:-20]:
    layer.trainable = False

# Keep BatchNormalization layers frozen
for layer in base_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

# Recompile with a lower learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
    loss='categorical_crossentropy',

    metrics=['accuracy']
)

print("Model ready for fine-tuning!")

# Step 21: Fine-tune MobileNetV2

fine_tune_history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=32,
    validation_data=(x_test, y_test)
)

print("Fine-tuning completed!")

# Evaluate the fine-tuned model
fine_tune_loss, fine_tune_accuracy = model.evaluate(
```

```
    x_test,
    y_test
)

print(
    f"Fine-tuned Test Accuracy: {fine_tune_accuracy * 100:.2f}%"
)

# Save fine-tuned model
model.save("mnist_fine_tuned_model.keras")

print("Fine-tuned model saved successfully!")
```

confusion_matrix.py

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.metrics import confusion_matrix

# Load your fine-tuned model
model = tf.keras.models.load_model(
    "mnist_fine_tuned_model.keras"
)

# Load MNIST test data
(_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Convert grayscale images to RGB
x_test = np.stack([x_test] * 3, axis=-1)

# Resize images to 32x32
x_test = tf.image.resize(x_test, [32, 32])

# Normalise for MobileNetV2
x_test = (x_test / 127.5) - 1.0

# Predict digit probabilities
predictions = model.predict(x_test, batch_size=32)

# Get predicted digit labels
y_pred = np.argmax(predictions, axis=1)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Display confusion matrix
plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")
plt.title("MNIST Transfer Learning Confusion Matrix")

plt.savefig("confusion_matrix.png", dpi=300, bbox_inches="tight")
plt.show()
```

sample_predictions.py

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

# Load the fine-tuned model
model = tf.keras.models.load_model("mnist_fine_tuned_model.keras")

# Load MNIST test data
(_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
# Keep original images for display
x_display = x_test.copy()

# Convert grayscale to RGB
x_test = np.stack([x_test] * 3, axis=-1)

# Resize to 32x32
x_test = tf.image.resize(x_test, [32, 32])

# Preprocess for MobileNetV2
x_test = (x_test / 127.5) - 1.0

# Predict classes
predictions = model.predict(x_test)
y_pred = np.argmax(predictions, axis=1)

# Show first 10 test images
plt.figure(figsize=(12, 6))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_display[i], cmap="gray")
    plt.title(f"True: {y_test[i]}\nPred: {y_pred[i]}")
    plt.axis("off")

plt.tight_layout()
plt.savefig("sample_predictions.png", dpi=300, bbox_inches="tight")
plt.show()
```

Script not found: app.py